以下是对《数据仓库与数据挖掘》试题的详细解答：

一、关联规则挖掘（Apriori 与 FP-Growth）

(a) Apriori 算法找出频繁项集

交易数据库：

顾客号	交易商品集合
1	{O, P, Q, R, S, T}
2	{O, P, R, S}
3	{O, P, S}
4	{O, P, Q}
5	{O, P}

最小支持度 = 60%，即出现次数 ≥ 3（5 条交易中至少出现 3 次）

步骤：

1. **C1（1-项集）：**

项集	支持度
O	5
P	5
Q	2
R	2
S	3
T	1

L1（频繁 1-项集）： {O}, {P}, {S}

2. **C2（2-项集）**：

项集	支持度
{O,P}	5
{O,S}	3
{P,S}	3

L2（频繁 2-项集）：{O,P}, {O,S}, {P,S}

3. **C3（3-项集）**：

项集	支持度
{O,P,S}	3

最终频繁项集：
{O}, {P}, {S}, {O,P}, {O,S}, {P,S}, {O,P,S}

(b) 找出所有关联规则

最小置信度 = 75%

从频繁项集 {O,P,S} 生成规则：

- O,P → S：支持度 = 3，置信度 = 3/5 = 60% ❌
- O,S → P：支持度 = 3，置信度 = 3/3 = 100% ✅
- P,S → O：支持度 = 3，置信度 = 3/3 = 100% ✅
- O → P,S：支持度 = 3，置信度 = 3/5 = 60% ❌
- P → O,S：支持度 = 3，置信度 = 3/5 = 60% ❌
- S → O,P：支持度 = 3，置信度 = 3/3 = 100% ✅

有效规则：

{O,S} → P

{P,S} → O

{S} → {O,P}

(c) FP-Growth 算法与 FP-tree

步骤：

1. 计算频繁 1-项集（同 Apriori）：{O:5}, {P:5}, {S:3}
2. 按支持度降序排序项：P, O, S
3. 按此顺序重排每条交易：

交易	重排后
1	P, O, S
2	P, O, S
3	P, O, S
4	P, O
5	P, O

4. **构建 FP-tree：**

```text
根
|
P:5
|
O:5
|
S:3
```

FP-tree 为一条链，因为所有交易都包含 P 和 O，且 S 出现在前三条交易中。

(d) Apriori 与 FP-Growth 比较

方面	Apriori	FP-Growth
思想	逐层生成候选集，剪枝	构建频繁模式树，无需候选集
扫描次数	多次扫描数据库	两次扫描
效率	候选集多时效率低	适合稠密数据集
组件化	生成候选 + 剪枝 + 计数	构建 FP-tree + 挖掘条件模式基

相同点：都用于挖掘频繁项集，支持度剪枝思想一致。
不同点：Apriori 产生大量候选，FP-Growth 通过树结构压缩数据，效率更高。

二、聚类分析（K-means）

(a) K-means 第二次迭代结果

点：A1(3,9), A2(2,7), A3(6,9), A4(1,3), A5(4,9), A6(2,7)
初始中心：A1(3,9), A3(6,9), A5(4,9)
距离函数：曼哈顿距离

第一次迭代聚类：

- 中心1(3,9)：A1(0), A2(4), A3(3), A4(8), A5(1), A6(4) → 最近：A1, A5
- 中心2(6,9)：A1(3), A2(6), A3(0), A4(11), A5(2), A6(6) → 最近：A3
- 中心3(4,9)：A1(1), A2(4), A3(2), A4(9), A5(0), A6(4) → 最近：A2, A4, A6

新中心：

- C1: A1(3,9), A5(4,9) → 中心 (3.5, 9)
- C2: A3(6,9) → 中心 (6, 9)
- C3: A2(2,7), A4(1,3), A6(2,7) → 中心 (1.67, 5.67)

第二次迭代聚类：

- 新中心1(3.5,9)：A1(1.5), A2(4.5), A3(2.5), A4(9.5), A5(0.5), A6(4.5) → 最近：A1, A3, A5
- 新中心2(6,9)：A1(3), A2(6), A3(0), A4(11), A5(2), A6(6) → 最近：无（被中心1抢走）
- 新中心3(1.67,5.67)：A1(6.34), A2(1.34), A3(7.34), A4(2.67), A5(6.34), A6(1.34) → 最近：A2, A4, A6

最终聚类：
C1: A1, A3, A5
C2: 无
C3: A2, A4, A6

中心更新：
C1: ((3+6+4)/3, (9+9+9)/3) = (4.33, 9)
C2: 无点，可删除或重新初始化
C3: ((2+1+2)/3, (7+3+7)/3) = (1.67, 5.67)

(b) K-means 缺点与改进

缺点：

- 对初始中心敏感
- 只能发现球形簇
- 对噪声和异常点敏感
- 需指定 k 值

改进算法：DBSCAN

- 基于密度，能发现任意形状簇
- 不需要指定 k
- 对噪声鲁棒

子序列定义

序列 A 是序列 B 的子序列，如果 A 的元素按顺序出现在 B 中，且每个元素是 B 中某个元素的子集（按项集序列匹配）。

<ad> 是否为子序列判断

- S10: <a(abc)(ac)d(cf)> → a 在第一个元素，d 在第四个元素 → ✅
- S20: <(ad)c(bc)(ae)> → a 和 d 在第一个元素中同时出现 → ✅（同一项集也算）
- S30: <(ef)(ab)(df)cb> → a 在第二个元素，d 在第三个元素 → ✅
- S40: <eg(af)cbc> → a 在第二个元素，但无 d → ❌

<ad> 的支持度：出现在 S10, S20, S30 → 3/4 = 75%

<(ab)> 的支持度：

- S10: a 在第一个元素，b 在第二个元素 → ✅
- S20: a 在第一个元素，b 在第三个元素 → ✅
- S30: a 在第二个元素，b 在第二个元素（同一项集） → ✅
- S40: a 在第二个元素，但无 b → ❌

支持度 = 3/4 = 75%

四、决策树算法（ID3）

建树过程：

1. 选择最优属性（信息增益最大）作为根节点
2. 按该属性取值划分数据集
3. 对每个子集递归建树，直到：
   - 所有样本属于同一类
   - 无属性可用 → 投票决定叶节点类别
   - 无样本 → 返回父节点多数类

决策过程：

从根节点开始，根据样本属性值沿树向下走到叶节点，叶节点的类别即为预测结果。

如果需要我将这些内容整理成答题卡或 PDF 格式，请告诉我。